

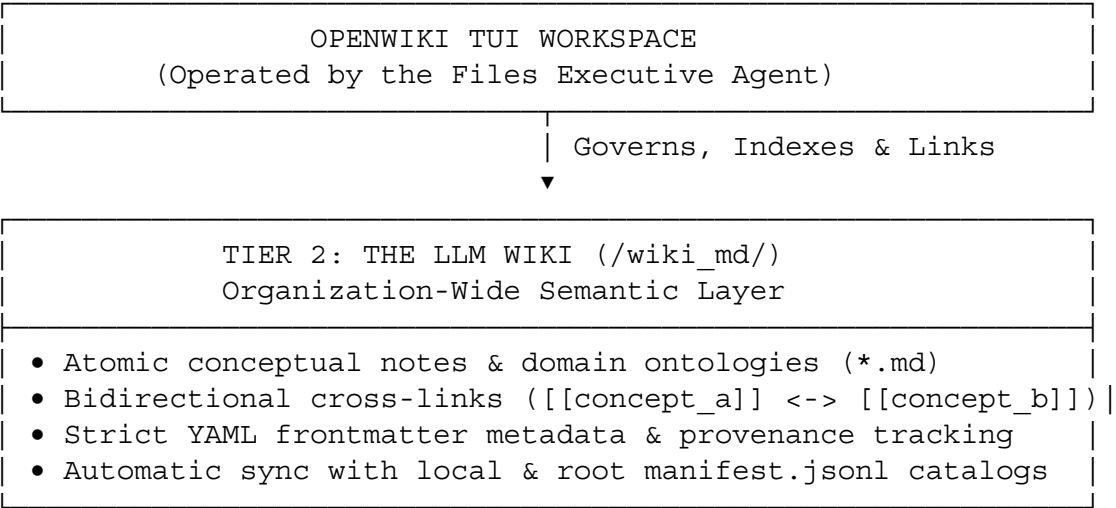
Here is the updated specification document. It establishes `wiki_md/` as the canonical **LLM Wiki Layer**, fully integrated with the **OpenWiki TUI** and governed by the **Files Executive Agent** as an organization-wide protocol.

SYSTEM SPECIFICATION: COGNITIVE REPOSITORY & AGENT ORCHESTRATION

Document Target: Autonomous Planning, Auditing, and Execution Agents **Version:** 2.1 (LLM Wiki Protocol & OpenWiki Executive Integration) **Core Framework:** 5+1 Tier Cognitive Memory, OpenWiki TUI File Management, Multi-Model Cross-Auditing, Sandboxed Red Gatekeeping, and Distributed RAG Manifests

1. The LLM Wiki Protocol & Files Executive Agent

The **LLM Wiki** (`wiki_md/`) serves as the organization's persistent semantic spine. It is governed directly by the **Files Executive Agent** operating within the **OpenWiki TUI**:



Protocol Responsibilities of the Files Executive Agent

- TUI-Driven Lifecycle Management:** The Files Executive Agent runs inside the OpenWiki terminal interface, monitoring file additions, moves, renames, and refactors across all tiers.
- Deterministic Linking & Backlinks:** Automatically verifies bidirectional Markdown links ([[entity_name]]), ensuring the LLM Wiki remains an intact graph without orphaned concepts.
- Extraction Intake Coordination:** Acts as the bridge between incoming raw sources in `01_raw_sources/`, synthesizing structured knowledge into `02_wiki_md/`, and passing

extracted procedural artifacts to 04_skills_runtime/.

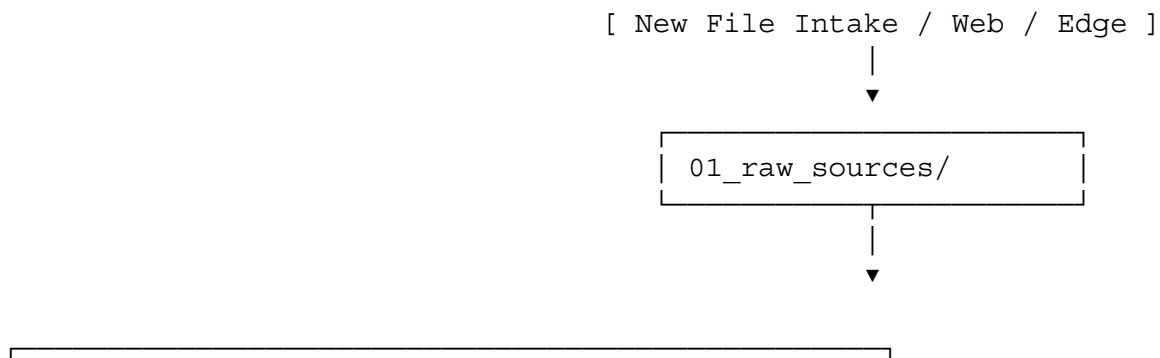
4. **Manifest Synchronization:** Whenever the agent modifies a node in wiki_md/, it immediately writes updated hashes, token counts, and entity tags to both 02_wiki_md/manifest.jsonl and the root manifest.jsonl.

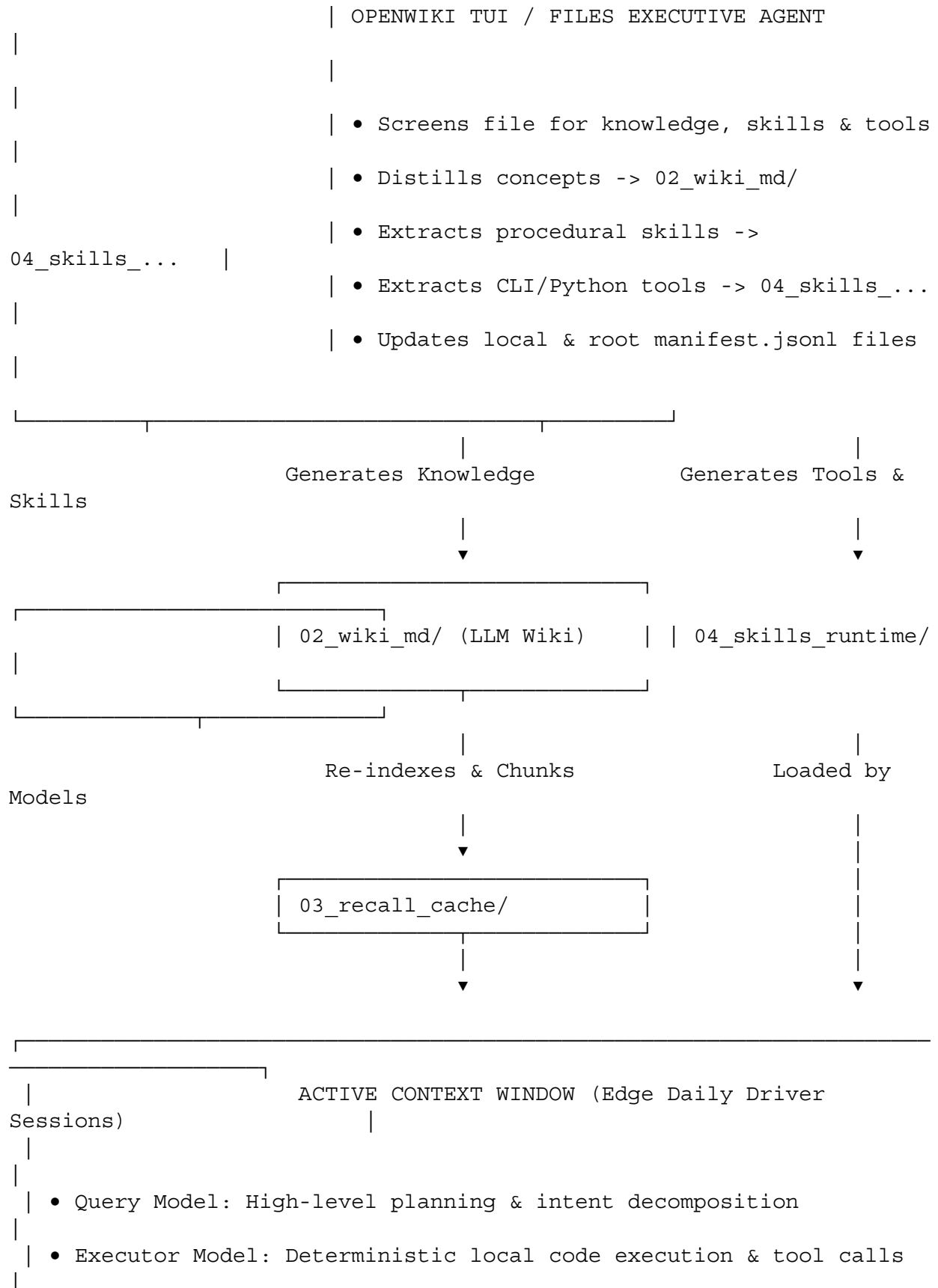
2. Updated Repository Topology (Featuring the LLM Wiki)

```
📦 vault_root/
├── 📄 MAP.md                                     # Master human-readable
index & ontology graph
├── 📊 manifest.jsonl                             # Root-level RAG
registry & global hash table
|
├── 📁 01_raw_sources/                           # TIER 1: COLD SENSORY
ARCHIVE (Read-Only Ground Truth)
|   ├── 📊 manifest.jsonl                       # Local RAG catalog of
all immutable raw sources
|   ├── 📄 pdf/                                  # Source whitepapers,
specs, data sheets (*.pdf)
|   ├── 🎬 media/                               # Video captures, audio
memos, transcripts (*.mp4, *.wav)
|   └── 📄 text/                                 # Raw text scrapes,
dumps, API payload exports (*.txt)
|
├── 📁 02_wiki_md/                               # TIER 2: THE LLM WIKI
LAYER (OpenWiki TUI Managed)
|   ├── 📊 manifest.jsonl                       # Local RAG catalog of
conceptual nodes & graph links
|   ├── 💡 concepts/                           # Atomic linked notes,
theory, and domain models (*.md)
|   ├── 🏛 architectures/                      # System blueprints,
data flows, interface specs (*.md)
|   ├── 🏷 entities/                           # Registries, schema
contracts, hardware profiles (*.md)
|   └── 🚫 indexes/                            # MOCs (Maps of Content)
and taxonomy clusters (*.md)
|
├── 📁 03_recall_cache/                          # TIER 3: WORKING MEMORY
ACCELERATOR (Rebuilt/Ephemeral)
|   ├── 📊 manifest.jsonl                       # Local RAG catalog of
vector & chunk shards
|   ├── ⚡ jsonl/                               # Pre-tokenized chunks
generated from wiki_md/
|   ├── 📐 vectors/                            # Dense vector indices
(FAISS/Chroma/HNSW checkpoints)
|   └── 🔑 kv_store/                           # Low-latency key-value
```

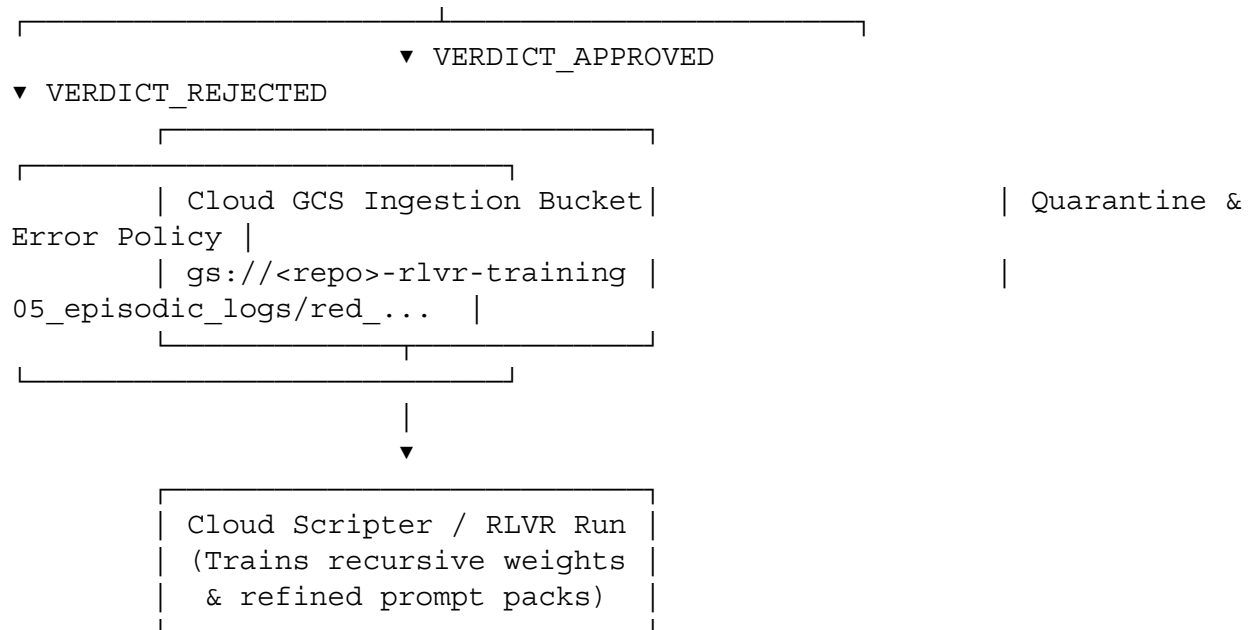
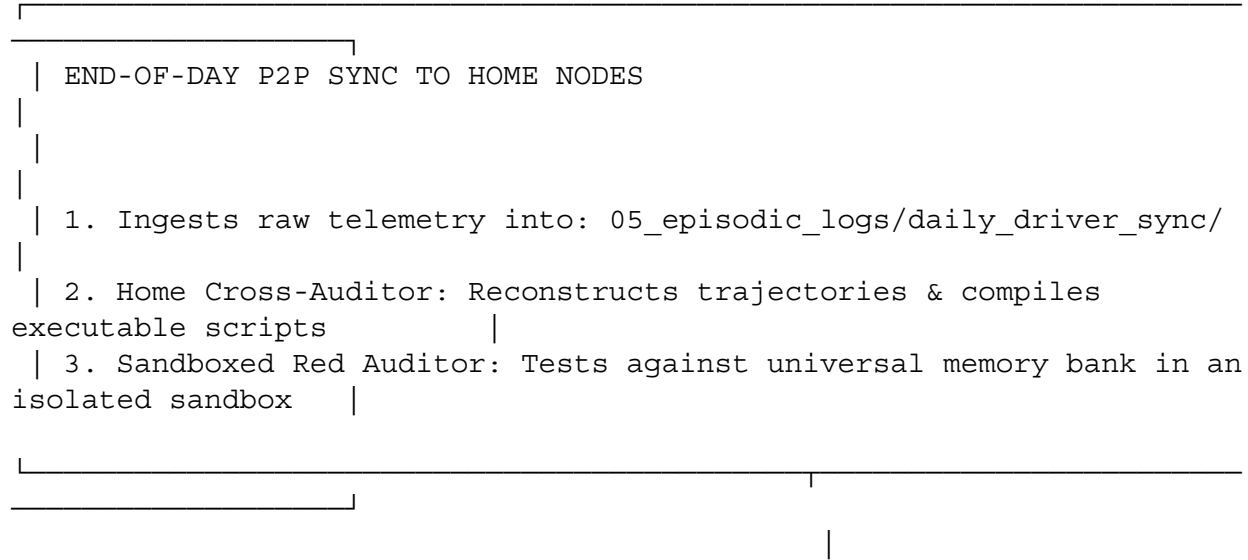
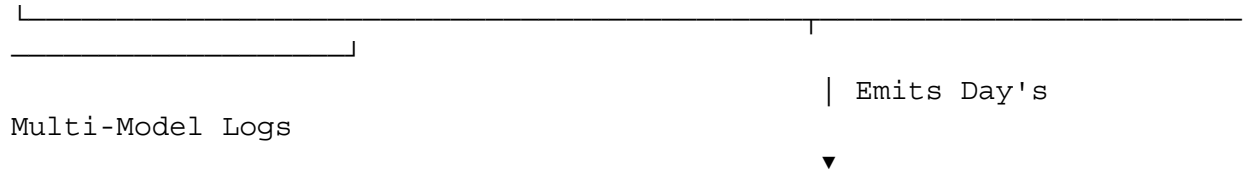
entity lookups	
— 📁 04_skills_runtime/	# TIER 4: PROCEDURAL
REPertoire & EXTRACTED TOOLS	
— 📊 manifest.jsonl	# Local RAG catalog of
all extracted skills & tools	
— 🎯 prompt_skills/	# Modular SKILL.md
prompt definitions (Extracted & Authored)	
— 🛠️ extracted_tools/	# Deterministic tools
mined by Files Executive Agent	
— cli/	# Extracted command-line
utilities (*.sh)	
— wrappers/	# Extracted API
interfaces & micro-functions (*.py)	
— ⚙️ runtimes/	# Master operational
scripts and execution hooks	
— 🛡️ policies/	# Self-correction
guidelines and validation schemas	
— 📁 05_episodic_logs/	# TIER 5: TELEMETRY,
AUDITS & TRAJECTORIES	
— 📊 manifest.jsonl	# Local RAG catalog of
sessions, audits, and verifiers	
— 📱 daily_driver_sync/	# Ingested logs from
edge device sessions	
— 🕒 trajectories/	# Multi-model traces
(Query, Executor, Frontier)	
— 🔴 red_audit_sandbox/	# Red Auditor evaluation
reports & pass/fail quarantine	
— ⚖️ rlvr_verifiers/	# Graded rewards,
assertion outcomes (+1.0 / -1.0)	
— 🧼 hygiene_reports/	# Schema integrity
audits, broken link checks	

3. End-to-End Orchestration & Interaction Flow





| • Frontier Model: Complex synthesis, high-order reasoning & edge
fallback |



4. LLM Wiki (wiki_md/) Document Standard

All documents created or maintained by the Files Executive Agent inside wiki_md/ must comply with this standard:

```
---
id: wiki_rag_indexing_protocol
tier: 2
type: architecture
created: 2026-09-03
updated: 2026-09-03
author: files_executive_agent
sources:
  - "01_raw_sources/pdf/rag_system_spec.pdf"
extracted_skills:
  - "04_skills_runtime/prompt_skills/manifest_sync.md"
extracted_tools:
  - "04_skills_runtime/extracted_tools/cli/build_manifest.py"
tags:
  - llm_wiki
  - openwiki
  - indexing
---
```

RAG Indexing Protocol

Context & Purpose

Summary of how the LLM Wiki synchronizes with the machine-readable cache...

Architectural Interfaces

The [[manifest_registry_spec]] defines how this node is indexed by the [[files_executive_agent]].

Linked Concepts

- [[hierarchical_manifest_routing]]
- [[procedural_tool_extraction]]

5. Master Execution Lifecycle for Planning & Executive Agents

1. Intake & TUI Triaging (Files Executive Agent):

- Inspect any incoming document in 01_raw_sources/ via OpenWiki TUI.
- Write core concepts into 02_wiki_md/.
- Extract tools to 04_skills_runtime/extracted_tools/ and prompt skills to 04_skills_runtime/prompt_skills/.

- Update the local 02_wiki_md/manifest.jsonl and root manifest.jsonl.
- 2. **Cache Compilation:**
 - Build pre-tokenized passages into 03_recall_cache/jsonl/ and refresh vector embeddings from updated wiki_md/ nodes.
- 3. **Session Execution:**
 - Plan and execute daily sessions across Query, Executor, and Frontier models.
 - Log all raw trajectories to 05_episodic_logs/.
- 4. **P2P Synchronization & Red Audit:**
 - At end-of-day, sync daily driver logs to home node via P2P.
 - Cross-auditor aligns model traces and extracts testable candidate scripts.
 - Red Auditor evaluates scripts in the local sandbox against the universal database.
- 5. **RLVR Cloud Hand-off:**
 - Push verified trajectories to the Google Cloud Storage bucket for cloud scripter processing and RLVR recursive model/prompt training.